

并行处理与体系结构(6) Parallel Processing and Architecture

2025-05-11

1

READING LIST (3)

ZY2457318	戚永飞
ZY2457328	于洁洁
ZY2457329	张超骏
ZY2457415	刘鑫

Topic 3

=====

Prefetching 第12周 (5月18日下午)

3a) T. Mowry et al. "Design and Evaluation of a Compiler Algorithm for Prefetching", Architectural Support for Programming Languages and Operating Systems, 1992.

3b) Y. Solihin et al. "Using a User-Level Memory Thread for Correlation Prefetching", International Symposium on Computer Architecture 2002.

▶ 2

2

READING LIST (4)

ZY2457308	王康宁
ZY2457311	王振伟
ZY2457312	王志飞
ZY2457316	文砚雷

Topic 4

=====

Synchronization 第12周 (5月18日下午/晚上?)

4a) J. Goodman et al. "Efficient Synchronization Primitives for Large Scale Cache-Coherent Multiprocessors", Architectural Support for Programming Languages and Operating Systems, 1989.

4b) J. Mellor-Crummey and M. Scott. "Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors", ACM Transactions on Computer Systems, 1991

▶ 3

3

READING LIST (5)

ZY2457213	李家兵
ZY2457216	李尚原
ZY2457229	邱文凯
ZY2457233	任星舟

Topic 5

=====

Multithreading 第13周 (5月25日上午)

5a) D. Tullsen et al. "Simultaneous multithreading: Maximizing On-Chip Parallelism", International Symposium on Computer Architecture 1995.

5b) D. Tullsen et al. "Exploiting Choice: Instruction Fetch and Issue on an Implementable Simultaneous Multithreading Processor", International Symposium on Computer Architecture 1996.

▶ 4

4

READING LIST (6)

Topic 6

=====

Multiple Processors on a Chip 第13周 (5月25日上午)

- 6a) K. Olukotun et al. "The Case for a Single-Chip Multiprocessor", Architectural Support for Programming Languages and Operating Systems, 1996
6b) G. Sohi et al. "Multiscalar Processors", International Symposium on Computer Architecture 1995.
6c) V. Krishnan and J. Torrellas. "A Chip Multiprocessor Architecture with Speculative Multithreading", IEEE Transactions on Computers 1999

ZY2457118	程晓
ZY2457124	段泽邦
ZY2457129	郭燕
ZY2457206	沈紫静

▶ 5

5

READING LIST (7)

Topic 7

=====

Speculative Parallelization and Execution 第13周 (5月25日下午)

- 7a) J. Steffan et al. "A Scalable Approach to Thread-Level Speculation" International Symposium on Computer Architecture 2000.
7b) J. Martinez et al. "Speculative Synchronization: Applying Thread-Level Speculation to Explicitly Parallel Applications", Architectural Support for Programming Languages and Operating Systems, 2002.

ZF2406102	盖一鸣
ZF2406108	罗晖
ZF2406114	张峻玮
ZY2457103	段欣然
ZY2457104	姜明月

▶ 6

6

回顾：预取

- ▶ 延迟容忍
- ▶ 在程序需要使用之前取数据，预测哪个地址会是未来需要的
- ▶ What: 预取什么地址
 - ▶ 预取无用的数据会浪费资源
 - ▶ 预取地址的准确预测是很重要的
 - ▶ 预取精度 = 有用的预取 / 发出的预取
 - ▶ 怎么知道预取什么
 - ▶ 基于过去访问模式的预测
 - ▶ 利用编译器关于数据结构的知识
 - ▶ 预取算法决定预取什么
- ▶ When: 何时发起预取请求
 - ▶ 预取太早，预取的数据可能在没被使用之前就被踢出暂存空间
 - ▶ 预取太迟，可能无法隐藏全部的访存延迟
 - ▶ 及时性指标
 - ▶ 更加的激进：尽量保持领先处理器访问流的幅度(硬件)
 - ▶ 在代码中更早的发起预取指令(软件)

▶ 7

7

回顾：预取

- ▶ Where: 预取的数据放到哪儿
 - ▶ 放到cache里
 - + 设计简单，不需要单独的缓冲
 - 可能会将有用的数据踢出 → cache污染
 - ▶ 放到独立的预取缓冲中
 - + 有用的数据被保护起来，不受预取影响 → 没有cache污染
 - 存储系统设计更加复杂
 - ▶ 很多现代系统将预取数据放入cache
 - ▶ 预取到哪个级别的cache?
 - ▶ 在cache里把预取的数据放到哪儿?
 - ▶ 预取的块和按需取的块同样对待吗?
 - ▶ 需要调整替换策略以使它能够更优待按需取的块吗?
 - ▶ 硬件预取器应该放在分层存储结构的什么位置?
 - ▶ 看到更复杂的访问模式:
 - + 可能有更好的预取精度和覆盖率
- ▶ 8 -- 预取器需要检验更多的请求(带宽密集, 更多输入端口的预取器?)

8

回顾：预取

► How: 软件、硬件、基于执行

► 软件预取

- 思路: 编译器/程序员将预取指令插入代码中合适的位置
- ISA 提供预取指令
- 程序员或编译器插入预取指令
 - 预取什么
 - 何时预取
- 通常只对“常规的访问模式”有效

► 9

9

回顾：预取

► How: 软件、硬件、基于执行

► 硬件预取

- 思路: 采用特殊的硬件观察load/store的访问模式, 基于过去的访问行为预取数据
 - + 可以协调地成为系统实现的一部分
 - + 不会浪费指令执行带宽
 - 为了检测模式会使硬件更加复杂
- 硬件监控处理器的存取
- 记录或者发现模式
- 自动生成预取地址
- Next-line 预取器 (也叫紧邻顺序预取器)
- 跨度(步长)预取器
 - 基于指令指针/程序计数器
 - 基于Cache块地址

► 10

✓ 流缓冲是基于cache块地址的跨度预取的一个特例; $N=1$

10

回顾：预取器性能

- 精度 (有用的预取 / 发出的预取)
- 覆盖率 (预取的缺失 / 所有的缺失)
- 及时性 (准时的预取 / 有用的预取)
- 带宽消耗
- Cache污染

► 预取器的激进特性影响所有的性能指标

- 激进程度取决于预取器的类型
- 对于大多数硬件预取器:
 - 预取距离: 领先访问需求流的距离
 - 预取度: 每个按需访问预取的数量
- 非常激进
 - + 更高的覆盖率, 更好的及时性
 - 很可能精度较低, 带宽消耗较高, cache污染也较高
- 非常保守
 - + 可能有较高的精度, 较低的带宽消耗, 较少的污染
 - 可能覆盖率较低, 不够及时
- 基于反馈的预取器调节
 - 动态监控预取器性能指标
 - 基于过去的性能状况调节预取器的激进程度并改变预取插入cache的位置

►

11

回顾：预取

► How: 软件、硬件、基于执行

► 预取不规则访问模式

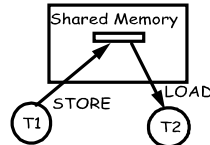
- 基于相关性的预取器
 - 马尔科夫预取
 - + 可以应对任意访问模式
 - 需要一张很大的相关表以获得高覆盖率
 - 及时性低
 - 消耗很多的存储带宽
 - 无法减少强制缺失
- 基于内容的预取器
 - 一种针对指针值的特殊预取器
 - 如何使基于内容的预取器更有效
- 基于预计算或预执行的预取器
 - 执行一个“线程”为主程序预取数据: 投机线程
 - 可以通过软件/程序员或者硬件生成
 - 在哪里执行预计算的线程?
 - 何时生成预计算线程?
 - 何时终止预计算线程?

► 12

12

同步

- ▶ 片上多处理器会共享部分内存
 - ▶ 通信隐式的通过LOAD和STORE实现
- 共享内存是一种通信机制



对处理器的相对速度不做假设
=> 在需要的时刻进行同步

无论只有一个副本还是有多个副本，同步都是需要的
同步与一致性无关，除非它们直接有交互

▶ 13

13

同步

- ▶ 需要“互斥”
- ▶ 假设下面对A的操作分别由两个线程T1和T2执行

```
T1      T2
A<-A+1  A<-A+1
```

- ▶ 程序员希望的是,无论两行程序的执行顺序如何,最终的结果都是A增加2
- ▶ 但是程序的语句的执行并不是原子的
- ▶ RISC架构的机器中,一条语句编译后的代码将包含若干条指令
- ▶ 一种可能的交错执行序是:

```
T1      T2
r1<-A    r1<-A
r1<-r1+1  r1<-r1+1
A<-r1    A<-r1
```

▶ 最终A将增加1 (不是期望中的2)

▶ 14

14

互斥

我们必须使程序语句的执行表现出原子性

- ▶ 临界区
 - ▶ 通过加锁和解锁原语来封装语句
 - ▶ 在离开临界区时,修改权被原子的“释放”
- ▶ 代码可能是这样的:

```
T1      T2
lock(La) lock(La)      /acquire
A<-A+1   A<-A+1
unlock(La) unlock(La)  /release
```

DEKKER 算法 (加锁)

- ▶ 假设A和B的初始值都是0

```
T1      T2
A:=1     B:=1      /acquire
while(B==1); while(A==1);
<critical section> <critical section>
A:=0     B:=0      /release
```

- ▶ 任一时刻最多只能有一个进程在临界区内
- ▶ 有可能发生死锁
- ▶ 复杂(解决死锁并且同步超过2个线程)

使用硬件原语同步

▶ 15

15

BARRIER 同步

在所有线程中实现全局同步

- ▶ 在任何一个线程被允许跨越BARRIER之前,所有的线程必须都抵达 BARRIER

```
P1      P2
...      ...
BAR := BAR+1; BAR := BAR+1;
while (BAR < 2); while (BAR < 2);
```

注意:需要一个临界区来增加 BAR

点到点同步

```
T1      T2
A = 1;   FLAG = 1;      /release
         /acquire
while (FLAG==0);
print A
```

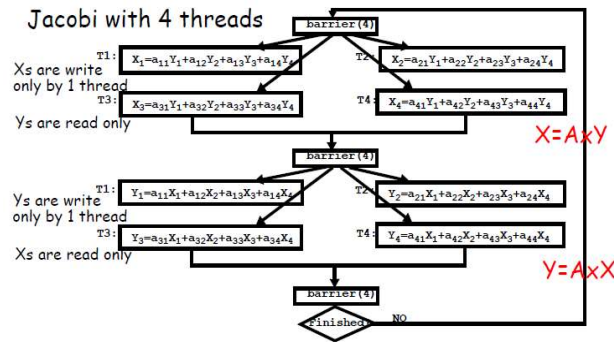
T1通过FLAG向T2发出信号(生产者/消费者同步)

- ▶ 注意:改写和读取FLAG时不需要临界区

▶ 16

16

例子



- 在第一阶段, 对 Xs 的访问是互斥的
- 在第二阶段, 对 Xs 有多次访问 (只读)
- 反过来对 Ys 也是如此

17

同步的方法

- 获取方法
 - 获取越过同步点的访问特权 (进入临界区, 跨越某些事件...)
- 释放方法
 - 使其它处理器获得越过同步点的访问特权
- 等待算法
 - 阻塞
 - 等待中的进程将不被调度
 - 开销很高
 - 允许处理器在某些情况下工作
 - 忙等
 - 等待中的进程不断重复测试某个位置直到那里的值改变
 - 开销低但是处理器被占用
 - 可能导致比较高的内存/网络通信量
- 忙等在某些时候更好
 - 调度开销高于期望的等待时间时

18

锁

- ISA 支持: 很多现代机器采用一种RMW的原子操作指令
 - IBM 370: 原子的比较和交换
 - X86: 任何指令都可以通过前缀加锁
 - SPARC: 交换
 - MIPS, PowerPC: 通过成对的指令支持
 - 加锁读LOAD-LOCKED, 有条件写STORE-CONDITIONAL

采用这些基本的机制构造软件锁

19

纯软件锁

```

Lock:    LW R2, lock
         BNEZ R2, Lock
         SW R1, lock      /R1 = 1
         RET

Unlock:  SW R0, lock
         RET
    
```

- 问题: 锁不是原子的—两个线程能同时获得锁
- 解决: 原子的读/写或交换指令
 - 原子地从某个位置读取值, 并且向该位置新值
- 最简单的一个: TEST_AND_SET (T&S)
 - T&S R1, lock
 - 读取 lock 进入 R1
 - 将 1 写入 lock
 - 如果返回R1的值是0则成功
 - 如果是1则失败

T&S 必须是原子的

20

软件锁

```
Lock:    T&S R1, lock
         BNEZ R1, Lock
         RET

Unlock:  SW R0, lock
         RET
```

▸ 其它的 R/M/W 原子操作

- SWAP R1, MEM_LOC: 交换 R1 和 MEM_LOC 的内容
- FETCH&OP
 - 例子: F&A (R1, MEM_LOC, CONST), 这里 CONST 是一个较小的值 (特殊情况下该值可以是 1)
 - 取回 MEM_LOC 放入 R1, 然后向 MEM_LOC 增加一个常数
- COMPARE&SWAP
 - CAS (R1, R2, MEM_LOC)
 - 比较 MEM_LOC 和 R1, 如果相等则交换 R2 和 MEM_LOC

▸ 21

21

降低T&S的频率

▸ 带退避的 T&S

- 增加延迟直到失败后的下一次重试
 - 例如, 指数退避
 - 在第 i 次重试时按照 $k \times c^i$ 退避

▸ TEST AND TEST&SET 锁

- 读取内存时检测
- 当标志位值变为 0, 尝试通过 T&S 获得锁
- 在有 CACHE 的结构下工作的很好

```
Lock:    LW R1, lock
         BNEZ R1, Lock
         T&S R1, lock
         BNEZ R1, Lock
         RET

Unlock:  SW R0, lock
         RET
```

▸ 22

22

加锁读和有条件写

```
T&S(Rx, lock): ADDI R1, R0, 1
               LL Rx, lock
               SC R1, lock
               BEQZ R1, T&S
               RET
```

▸ 加锁读或关联读 (LL)

- LL 读取 lock 的值放入寄存器 Rx

▸ 有条件写 (SC)

- 尝试将 1 写入 lock
- 当 LL 之后没有其它线程写入 lock 时, SC 成功
 - 如果 SC 成功则指令序列 LL-SC 是原子的
 - 如果 SC 失败, 它无法完成写入内存操作, 会将 R1 设置为 0

▸ 当发生以下情况时一个 SC 会失败

- 它检测到 LL 之后有写 lock 的操作插入
- 尝试获得总线使用权, 但是另一个 SC 抢先获得总线

▸ 两个优点

- 在流水线中更容易实现
- 灵活

▸ 23

23

加锁读和有条件写

▸ 更有想象力的原子操作可以通过在 LL 和 SC 之间增加代码来实现, 但是

- 代码要足够短才能使 SC 更有希望成功
- 要避免不能回退的指令 (比如, STORE, 会导致异常的指令)

▸ 实现

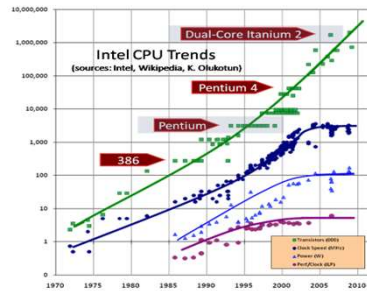
- 当 LL 执行时置位 LL-BIT
- 在总线接口上监听更新或置无效信号并相应重置 LL-BIT
- SC 检测 LL-BIT, 如果 LL-BIT 被重置则失败
- LL-BIT 可以只有一位, 也可以有若干位
 - 可以为所有地址设置一位 LL-BIT
 - 可以将地址关联到 LL-BIT
 - 也可以为多个不同地址设置多个 LL-BIT

▸ 24

24

单处理器的问题

- ▶ 复杂性
- ▶ 新增硬件的边际效用
- ▶ 能耗
- ▶ 缺少指令级并行

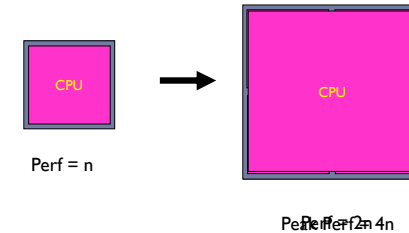


▶ 25

25

性能提升的真相

- ▶ 性能并不是按照晶体管数量线性增长的
 - ▶ $\text{Perf} \sim O(n^{1/2})$



▶ 26

26

两种选择

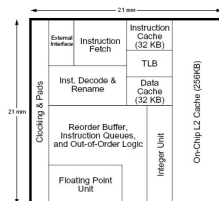


Figure 2. Floorplan for the six-issue dynamic superscalar microprocessor.

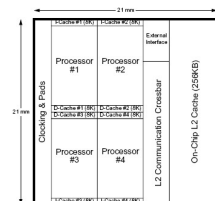


Figure 3. Floorplan for the four-way single-chip multiprocessor.

- ▶ [Olukotun 1996] Multithreading Multicore (Chip Multiprocessing)

▶ 27

27

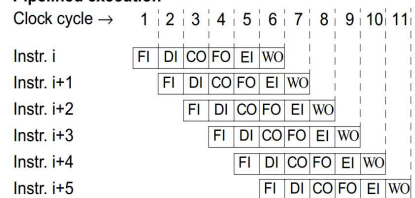
并行的分类

- ▶ 指令级并行 (ILP)
 - 独立的指令之间
 - 硬件只能通过指令窗口观察到ILP
 - 编译器可以重新组织指令以使得落入指令窗口的指令相对独立
- ▶ 线程级并行 (TLP)
 - 编译器将程序切分成多个控制线程 (每个线程执行一组指令)
 - 不需要一个大的窗口 (每个线程可以关注一个小的窗口)

▶ 28

28

Pipelined execution



流水线(Pipeline)——利用执行指令所需的操作之间的并行性,实现多条指令重叠执行的一种技术。

就像装配线那样,不同的步骤并行完成流水线中不同指令的不同部分。每一个步骤称为一个**流水节拍**或一个**流水段**。

▶ 29

29

Superpipelined execution

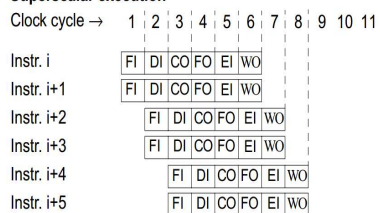


超流水线(SuperPipeline)——进一步细化流水线中的每个流水段,使得指令间拥有更高的并行程度。

▶ 30

30

Superscalar execution

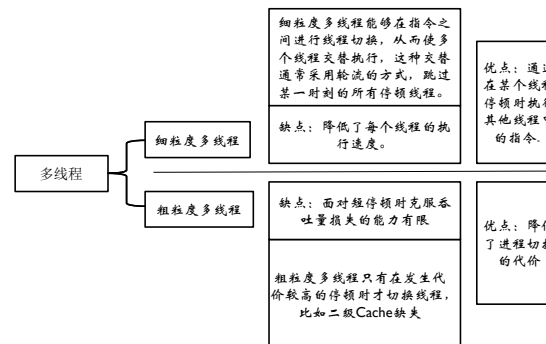


超标量体系结构(Superscalar)——CPU在一个时钟周期可以同时发射(issue)多条指令在不同的执行单元中执行,这就实现了指令级的并行。

超标量处理器的内核中一般有多个执行单元(或称功能单元),如算术逻辑单元、寄存器单元、乘法器等。

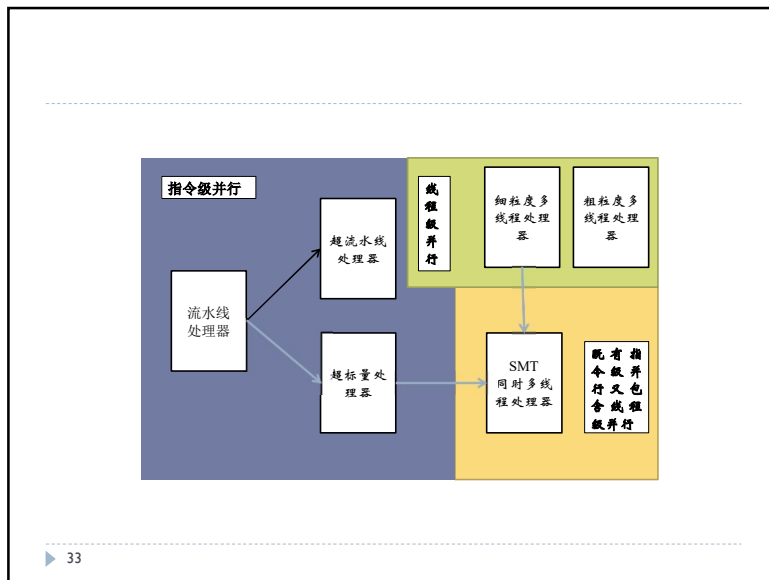
▶ 31

31

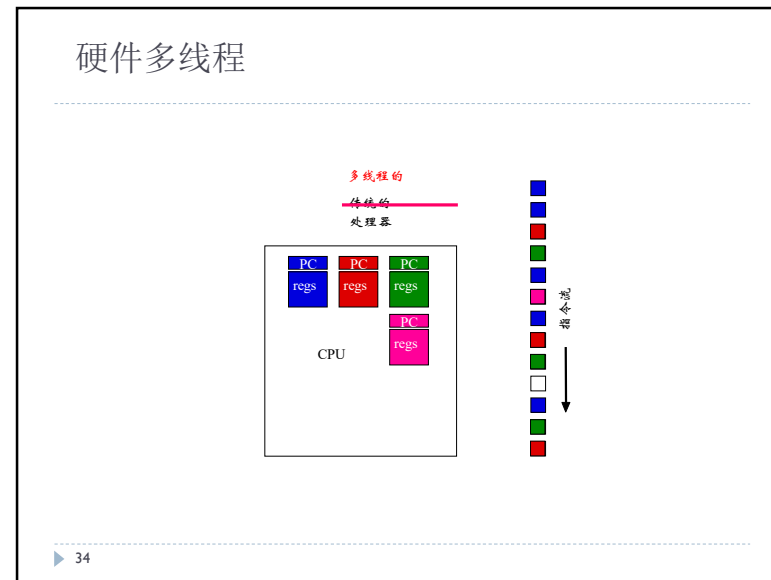


▶ 32

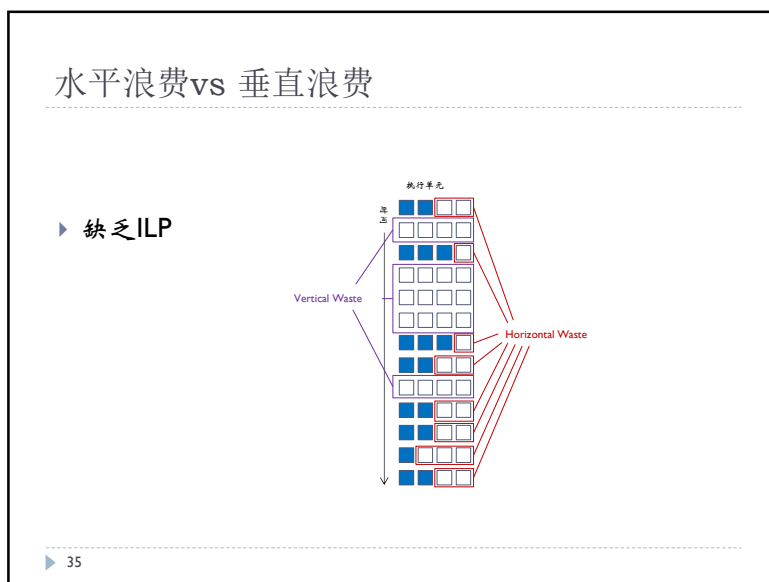
32



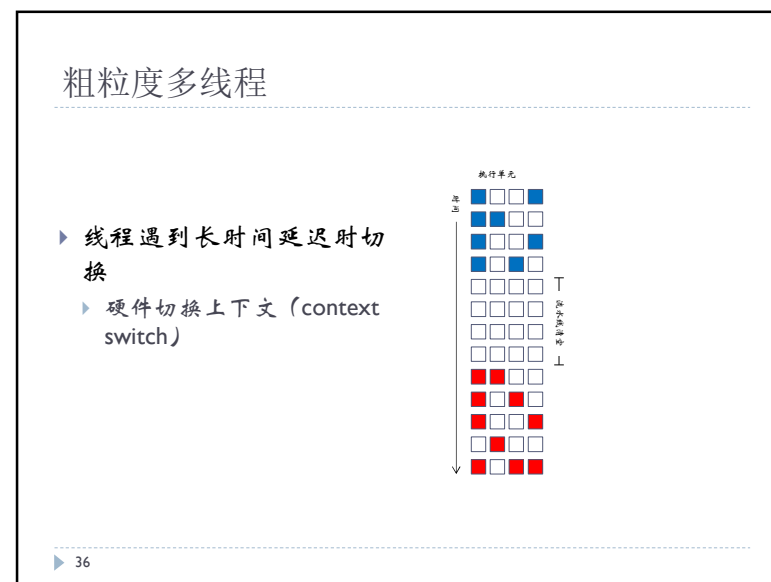
33



34



35



36

细粒度多线程

- 线程轮流切换
- 流水线中存储硬件上下文



▶ 37

37

同时多线程

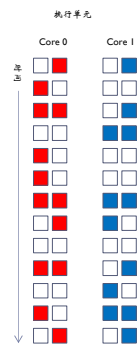
- 解决两种浪费



▶ 38

38

片上多处理器

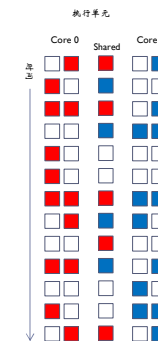


▶ 39

39

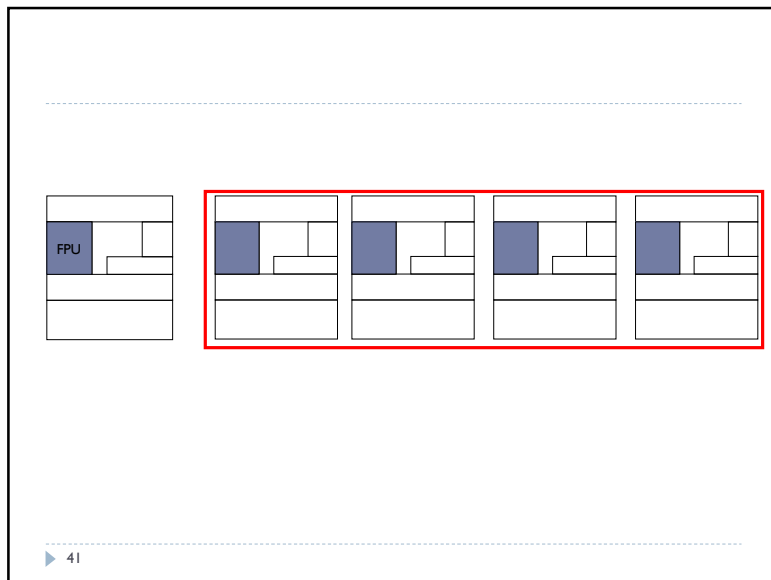
连体核体系结构

- “Conjoined-Core Chip Multiprocessing,” Rakesh Kumar, Norman P. Jouppi, Dean M. Tullsen, In 37th International Symposium on Microarchitecture, December, 2004.

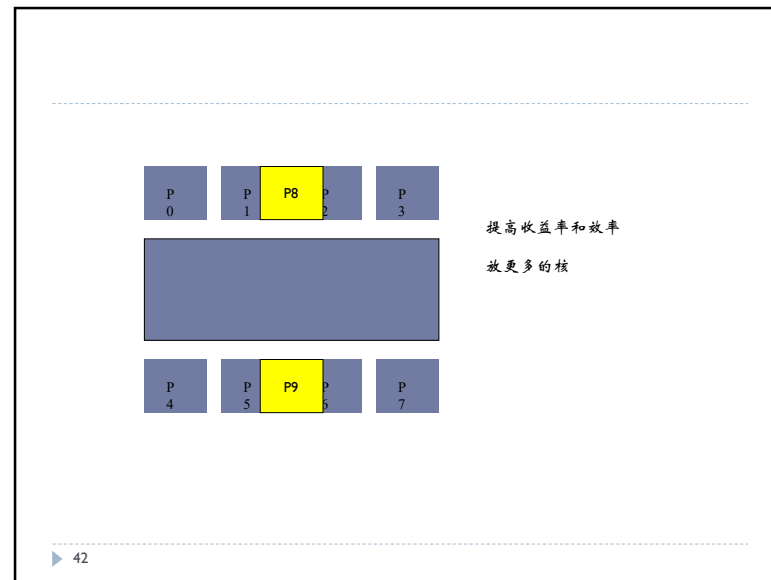


▶ 40

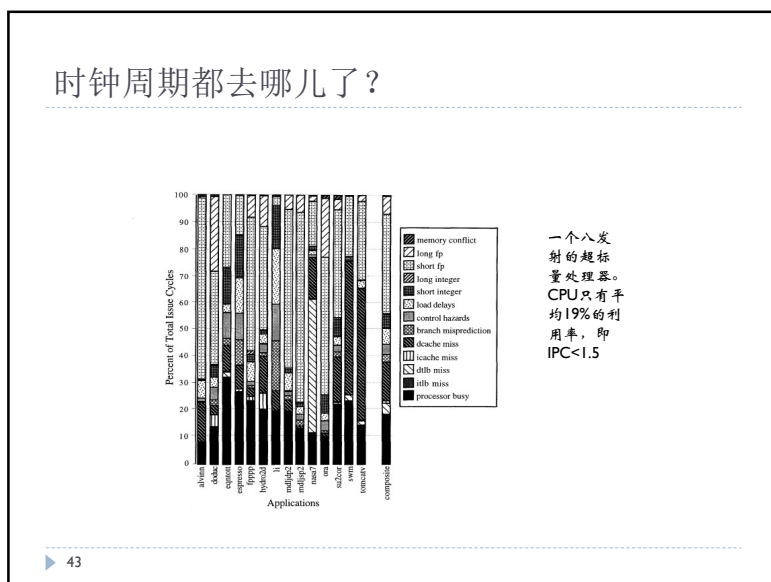
40



41



42



43

片上多处理器的动机

- ▶ 芯片集成度（密度）不断增加
- ▶ 动态超标量方法
 - ▶ 单线程控制的局限
 - ▶ 大发射窗口
- ▶ 芯片面积的优化利用
 - ▶ SMT
 - ▶ CMP

44

超标量

- ▶ 多指令发射，动态调度，投机执行，非阻塞cache
- ▶ 趋势
 - ▶ 更宽的指令发射
 - ▶ 更多的投机执行
- ▶ 限制
 - ▶ 指令级并行的数量
 - ▶ 电路的限制
- ▶ 线程级并行（TLP）

45

SMT的动机

- ▶ 宽发射的超标量结构仍然可行
 - ▶ 有必要继续提升单一程序的性能
- ▶ 多道程序共享取指令和发射带宽
 - ▶ TLP改善了宽发射超标量的吞吐
 - ▶ 仍然可以利用单一程序中较高的ILP

46

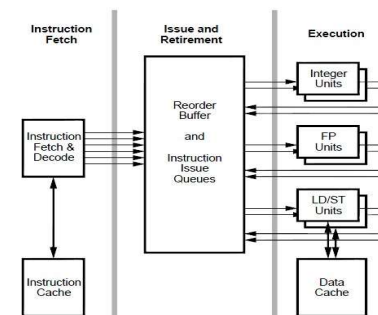
CMP（多核）的动机

- ▶ 宽发射超标量并非解决之道
 - ▶ 很高的时钟频率VS很小的IPC增加
 - ▶ 设计和验证的复杂性
- ▶ 多个简单的处理器
 - ▶ 利用TLP
 - ▶ 适度的ILP加上高频率时钟

47

动态超标量CPU

- ▶ 指令队列通常被设计成针对不同类型指令的多个队列
- ▶ 流水段
 - ▶ 取值
 - ▶ 发射
 - ▶ 执行



48

取指

- ▶ 目标：实现大指令窗口
- ▶ 限制
 - ▶ 预测错误的分支
 - ▶ 分支预测buffer
 - ▶ 指令未对齐
 - ▶ 对指令译码器中的指令进行对齐
 - ▶ Cache缺失
 - ▶ 缺失率严重影响发射窗口

取指阶段不是瓶颈

49

发射

- ▶ 重命名的指令会插入指令发射队列
 - ▶ 操作数就绪即可开始发射指令
 - ▶ 重命名通过RAT、指令队列和重排序缓冲器实现
 - ▶ 指令进入发射队列随时更新指令之间的依赖信息
 - ▶ 需要很多比较器电路
 - ▶ 发现相互独立的指令
 - ▶ 在大的指令队列里广播标志位信息
- 指令发射队列会限制时钟周期

50

执行

- ▶ 操作数来自寄存器堆或者前序指令的旁路
- ▶ 宽发射超标量结构的问题
 - ▶ 寄存器堆
 - ▶ 如何容纳更多的重命名寄存器？
 - ▶ 更多的输入输出端口？
 - ▶ 复杂性，与端口数平方成正比，与发射宽度平方成正比
 - ▶ 旁路逻辑
 - ▶ 复杂性，与执行单元平方成正比
 - ▶ 组合逻辑延时
 - ▶ 功能组件
 - ▶ 需要更多的与数据cache的连接

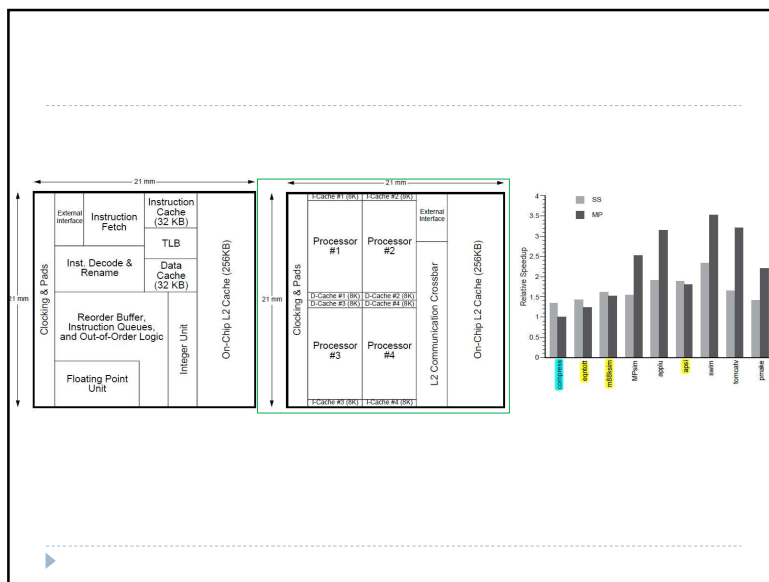
限制指令窗口

51

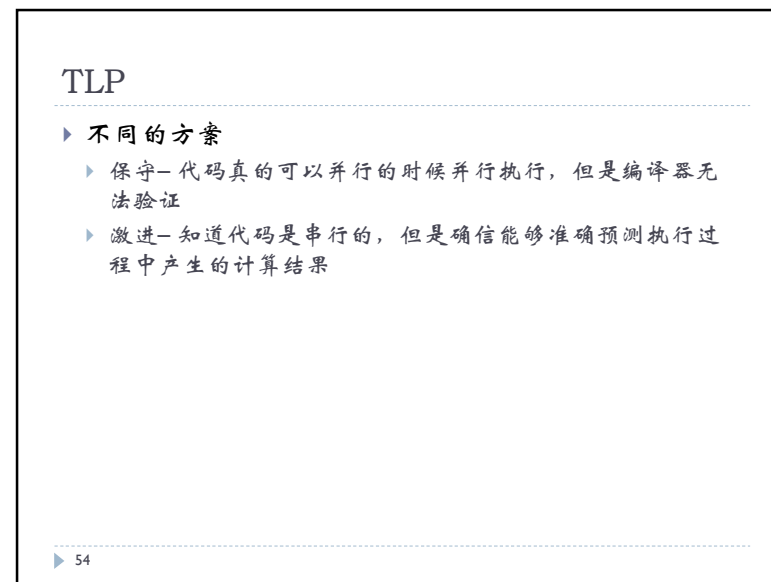
单芯片多处理器

- ▶ 技术推动
 - ▶ 发射宽度带来的收益是受限的
 - ▶ 非集中式微体系结构：构建多个简单的快处理器比构建一个复杂的处理器容易得多
- ▶ 需求牵引
 - ▶ 应用的并行粒度差别很大
 - ▶ 每时钟周期少于10条指令（一些整型运算的应用）
 - ▶ 每时钟周期多于40条指令（浮点运算为主的应用，基于循环）

52



53



54

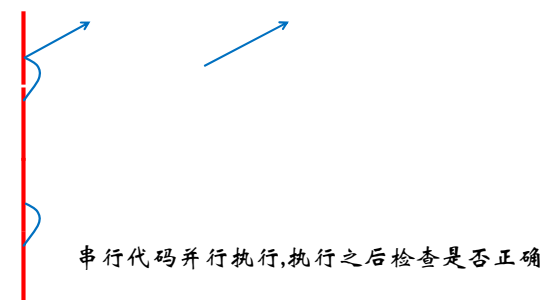
TLS的动机—保守的并行性

- ▶ 除非100%的安全否则没有并行
- ▶ 访问模式很难分析
 - ▶ 带下标的数组下标
 - ▶ 指针
- ▶ 大多数并行代码中都会出现的一些“极端个案”

▶ 55

55

TLP (SpMT) – 主要思路



串行代码并行执行,执行之后检查是否正确

▶ 56

56

TLP – 如何解决烦人的依赖性?

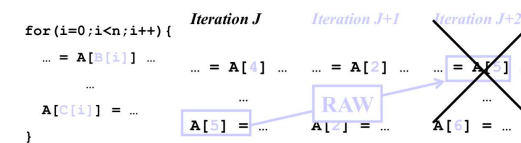
- ▶ 等
- ▶ 投机, 必须的情况下重新执行

▶ 57

57

投机并行

- ▶ 投机的并行执行不容易分析的代码
- ▶ 假设没有依赖性, 并行执行
- ▶ 跟踪访存行为, 检测冲突
- ▶ 清除并重启冲突的线程
- ▶ 随时维持安全线程 (最早开始的) 的执行



▶ 58

58

TLS (线程级投机)

- ▶ 主要思路: 并行的执行不安全的线程
 - ▶ 通常是来自串行程序中的线程
 - ▶ 硬件监控数据冲突
 - ▶ 冲突发生时, 清除并重启相关线程
 - ▶ 不发生冲突
 - ▶ 线程执行完毕
 - ▶ 一个线程前序线程都执行完毕, 该线程变为非投机或者安全线程
 - ▶ 前序线程全部提交后, 该线程提交
- 线程按序提交

▶

59

内存通信

- ▶ 可能导致线程崩溃
- ▶ 对编译器不可见
- ▶ Load可投机执行
 - ▶ 可以从最近的前序获得
 - ▶ 保存读取的数据
- ▶ Store可投机执行
 - ▶ 投机时缓冲更新
 - ▶ 检查后序是否有可能提前load
 - ▶ 如果有, 清除后序和冲突线程
- ▶ 线程提交后, 安全状态返回给内存, 数据写回

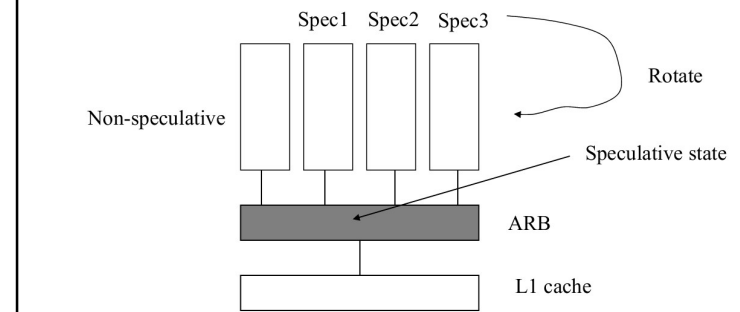
▶

60

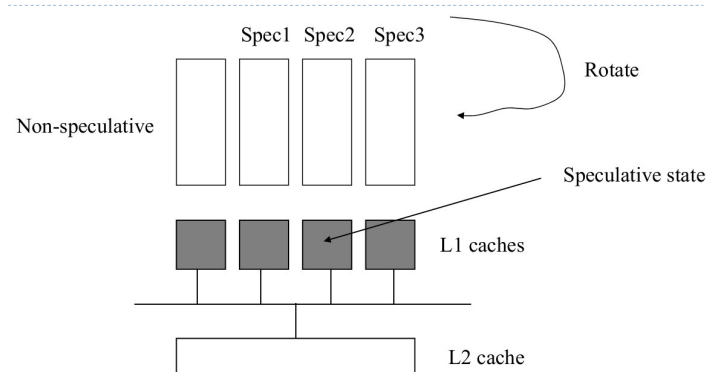
依赖冲突

- ▶ 依赖冲突的类型：乱序
 - ▶ Load, Store: 名字依赖, 硬件可解决
 - ▶ Store, Store: 名字依赖, 硬件可解决
 - ▶ Store, Load: 真依赖, 可导致清除
- ▶ 解决名字依赖: “多版本”支持
 - ▶ 投机或非投机store到同一位置, 创建一个新的版本
- ▶ 保持并检查投机版本
 - ▶ 集中模式: ARB (Address Resolution Buffer)
 - ▶ 非集中模式: LI 或者写缓冲器

61



62



63

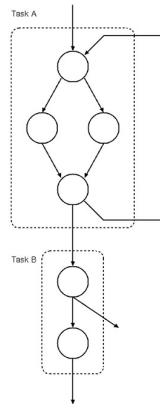
多标量

- ▶ 利用串行程序中的“隐式”线程级并行
- ▶ 编译器将程序划分成任务
- ▶ 任务在独立的进程资源中调度
- ▶ 硬件处理任务间的寄存器依赖
- ▶ 内存依赖通过内存投机的机制解决

64

任务

- ▶ 一个任务是整个控制流图（CFG）的一个子图
- ▶ 编译器选择任务并传递给硬件
- ▶ 处理器预测并调度任务
- ▶ 任务间可能有数据或控制依赖

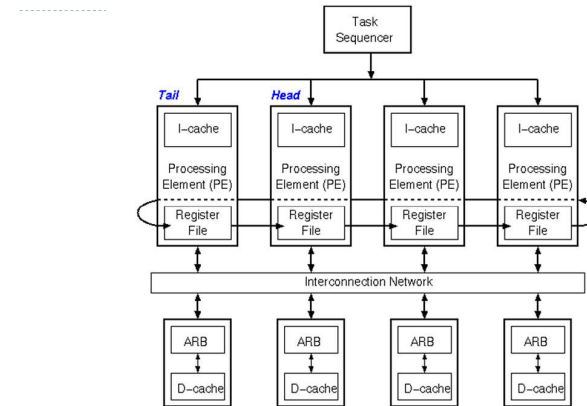


65

编译器的任务

- ▶ 选择任务——对CFG进行分区
 - ▶ 负载均衡
 - ▶ 最小化任务间数据依赖
 - ▶ 最小化任务间控制依赖
 - ▶ 任务内嵌入难以预测的分支
- ▶ 传递信息给执行机构
 - ▶ 任务的头部（header）
 - ▶ create_mask，指明该任务可能修改的所有寄存器
 - ▶ 后续任务的PC
 - ▶ 插入必要的特殊指令

67

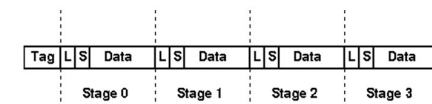


66

地址解析缓冲器（ARB）

- ▶ 多标量发射当地址计算完成后立即load ARB/D-Cache
- ▶ 乐观投机：没有对相同地址未完成的前序store操作
- ▶ ARB像一个cache，保存着所有未完成的读写状态

ARB实现内存重命名



68

投机的同步

- ▶ 加速比
- ▶ 关键路径不受影响
- ▶ 投机缓存溢出容易控制
- ▶ 硬件简单
- ▶ 不需要特殊的编程开销
- ▶ 改进的余地
 - WAR, WAW 依赖
 - 伪共享

69

TLS的评价——一种观点

- ▶ TLS 是非常有前途的技术
- ▶ TLS 的安全线程机制是有效的

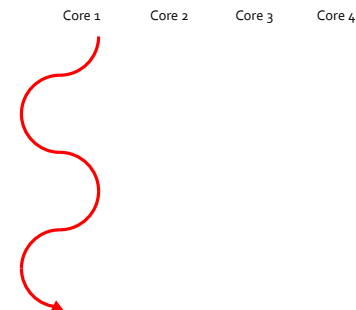
70

非传统的并行

- ▶ 并行性—使用多个上下文获得好于一个上下文所能获得的性能
- ▶ 传统的并行性—使用附加的线程/处理器分配计算：线程分割执行流
- ▶ 非传统的并行性—附加线程用于加速计算，不需要分配原始计算
 - ▶ 最大的优势：几乎对所有代码，不管它的可并行性如何，都能从并行中获益
 - ▶ 其他的优点：可以灵活地增加或减少线程而不会导致明显的执行中断

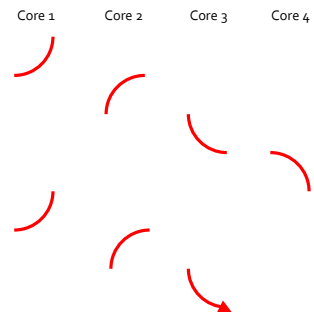
71

传统的并行



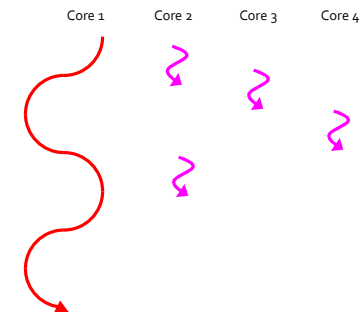
72

传统的并行



73

非传统的并行(一个例子)



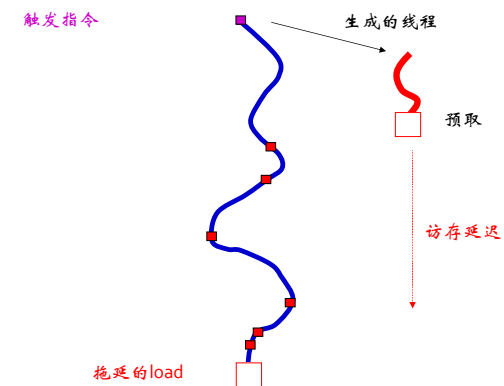
74

Helper 线程

- ▶ Helper线程预取
 - ▶ 由程序员/编译器生成
 - ▶ 由硬件生成
- ▶ Helper线程改善代码质量(重编译)

75

投机预计算



76

对比传统预取的优势

- ▶ 投机预计算使用的是真实的程序代码，可以预计算没有可预测模式的地址信息
- ▶ 投机预计算运行在独立的线程，比软件预取对主线程的干扰要小，如果它的效果不好，可以直接kill掉
- ▶ 同时，由于与主线程解耦，预取器不受主线程控制流约束
- ▶ 当主线程停顿的时候预取可以继续执行(可能甚至更快)

77

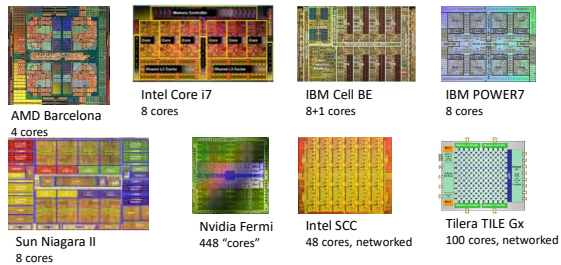
回顾：紧耦合多处理器的主要难点

- ▶ 共享存储同步
 - ▶ 锁, 原子操作
- ▶ Cache 一致性
- ▶ 访存操作的序
 - ▶ 程序员希望硬件提供什么?
- ▶ 资源共享, 竞争和分区
- ▶ 通信: 互连网络
- ▶ 负载均衡

78

片上众核

- ▶ 比一个大核更简单并且功耗更低
- ▶ 片上大规模并行



79

对于片上多核

- ▶ 我们想要:
 - ▶ 当我们在N个核上并行化一个应用，我们能获得N倍在单个核上的性能
- ▶ 我们能够得到:
 - ▶ Amdahl定律 (串行瓶颈)
 - ▶ 并行部分的瓶颈

80

回顾：并行性

▶ Amdahl定律

- ▶ f: 程序中可并行的部分
- ▶ N: 处理器个数

$$\text{加速比} = \frac{1}{1-f + \frac{f}{N}}$$

- ▶ Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

▶ 最大加速比受限于串行部分: 串行瓶颈

▶ 并行部分通常不完美

- ▶ 同步开销 (比如, 对共享数据的更新)
- ▶ 负载不均衡开销 (并行化不完美)
- ▶ 资源共享开销 (N个处理器之间的竞争)

81

问题: 串行化的代码段

▶ 很多并行程序无法完全并行化

▶ 串行化代码段的产生

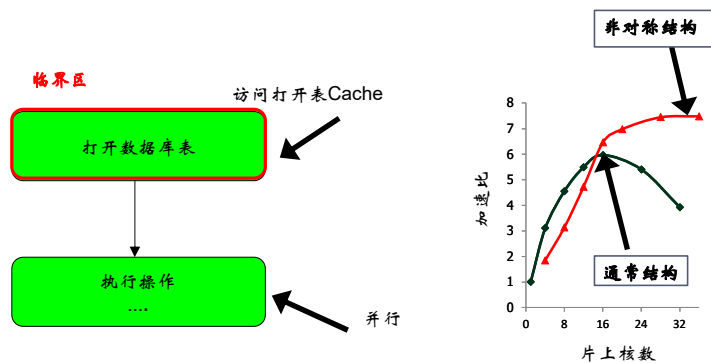
- ▶ 连续的部分(Amdahl的“串行部分”)
- ▶ 临界区
- ▶ 栅障
- ▶ 流水化程序中的受限阶段

▶ 串行化的代码段

- ▶ 降低性能
- ▶ 限制扩展性
- ▶ 浪费能源

82

MySQL的例子



83

不同代码段的需求

▶ 我们想要:

▶ 串行代码段 → 一个强有力的“大”核

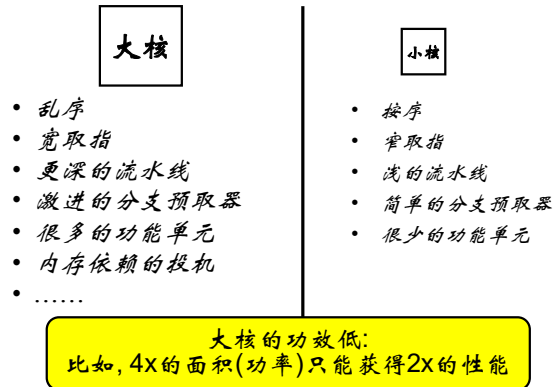
▶ 并行代码段 → 许多弱的“小”核

▶ 这两者互相冲突:

- ▶ 如果你有一个强有力的核, 就不可能同时拥有很多核
- ▶ 一个小核在能耗和面积消耗方面都远比一个大核更高效

84

“大” vs. “小” 核



85

大核 vs. 小核

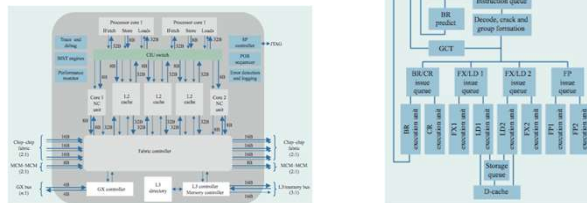
- ▶ Grochowski et al., “Best of both Latency and Throughput,” ICCD 2004.

	Large core	Small core
Microarchitecture	Out-of-order, 128-256 entry ROB	In-order
Width	3-4	1
Pipeline depth	20-30	5
Normalized performance	5-8x	1x
Normalized power	20-50x	1x
Normalized energy/instruction	4-6x	1x

86

大核: IBM POWER4

- ▶ Tendler et al., “POWER4 system microarchitecture,” IBM J R&D, 2002.
- ▶ 另外一种对称多核芯片...
- ▶ 但是, 更少、更强有力的核



87

IBM POWER4

- ▶ 2个核,乱序执行
- ▶ 每核100-entry的指令窗口
- ▶ 8宽度取指、发射和执行
- ▶ 大容量,本地+全局混合分支预测器
- ▶ 1.5MB, 8路 L2 cache
- ▶ 基于流的激进预取

88

谢谢!